

AeroWMS: Technical Architecture & Interview Defense Guide

12 Key Engineering Questions & In-Depth Technical Explanations

Candidate Name: **Izza Faris**

Role Focus: Full Stack / Java Spring Boot & Angular

Candidate Email: izzafaris.it@gmail.com

GitHub Profile: github.com/izza-faris

Project Repository: github.com/izza-faris/WMS-SYSTEM

Live Web Portal: izza-faris.github.io/WMS-SYSTEM

Q1 Why did you choose this technology stack?

AeroWMS was engineered with a production-grade, highly maintainable cloud architecture:

- **Java 21 LTS & Spring Boot 3.3:** Provides enterprise-grade compile-time type safety, preventing runtime errors in mission-critical financial and inventory calculations. Spring Security 6 provides battle-tested authorization, while Java 21 Virtual Threads ensure high throughput during heavy concurrent scanning operations.
- **Angular 18 (Standalone & Signals):** Chosen over lightweight UI libraries for its strict architectural structure, dependency injection, and typed reactive forms. Angular Signals provide fine-grained reactivity for live telemetry without full-page re-renders.
- **MongoDB Atlas Cloud Database:** Warehouse inventory involves heterogeneous catalog items (perishables require expiry dates, electronics need serials, bulk items require volumetric dimensions). A document model allows dynamic product attributes without complex relational schema migrations, while compound indexes ensure sub-millisecond multi-tenant queries.
- **Google ZXing & html5-qrcode:** Eliminates expensive proprietary barcode scanner hardware by enabling any smartphone camera, tablet, or plug-and-play USB laser scanner gun to operate seamlessly.

Q2 How do Authentication and Authorization work?

AeroWMS implements completely stateless, token-based security governed by Spring Security 6:

```
[Client Login] → POST /api/v1/auth/login → [BCrypt Hash Validation] → [Issue Signed JWT]
```

- **Authentication:** When credentials are submitted, AuthService fetches the user document and validates passwords using BCryptPasswordEncoder (salted, cost factor 10). Upon success, JwtTokenProvider generates an HMAC-SHA256 signed JWT containing the user's ID, email, role, clientId, and branchId.
- **Authorization & RBAC:** 4 roles are enforced: PLATFORM_ADMIN, CLIENT_ADMIN, WAREHOUSE_MANAGER, and WAREHOUSE_STAFF. SecurityConfig enforces URL route-level access rules (e.g. `hasRole('PLATFORM_ADMIN')`).
- **Branch Boundary Enforcement:** `TenantSecurityService.validateBranchAccess()` ensures Managers and Staff can only view and modify data within their assigned branch, while CLIENT_ADMIN has tenant-wide visibility.

Q3 How is JWT implemented?

Implemented using `io.jsonwebtoken (jjwt 0.12.6)` with strict stateless request interception:

- **Token Claims Structure:** Claims encode `sub` (email), `userId`, `clientId` (tenant), `branchId`, and `role`. Signed with a 256-bit cryptographically secure key with a 24-hour expiration window.
- **The Filter Pipeline (`JwtAuthenticationFilter`):**
 1. Intercepts incoming requests and strips the `Bearer` prefix from the `Authorization` header.
 2. Cryptographically verifies signature authenticity and timestamp validity.
 3. Extracts claims and populates the Java `ThreadLocal` context:

```
TenantContext.set(new TenantContext.TenantInfo(userId, clientId, branchId, role, email));
```

4. Builds a `UsernamePasswordAuthenticationToken` with authority `ROLE_<ROLE>` and registers it in Spring's `SecurityContextHolder`.
5. **Memory Leak Protection:** In a `finally` block, `TenantContext.clear()` is executed on every request to prevent `ThreadLocal` retention when threads return to the Tomcat thread pool.

Q4 How does Multi-Tenancy work under the hood?

AeroWMS uses **Logical Document Partitioning with ThreadLocal Scoping**:

```
HTTP Request → [JwtAuthenticationFilter] → TenantContext.set(clientId) → Repositories: findById(...)
```

- **Scoped Document Storage:** Every business collection (products, inventory, warehouses, transactions) contains a mandatory indexed `clientId` field.
- **ThreadLocal Scoping:** `TenantContext.getCurrentClientId()` injects the authenticated client ID into domain services without requiring it as an untrusted method parameter.
- **IDOR Protection:** If an authenticated user from Tenant A tries to manipulate an entity belonging to Tenant B by guessing an ID (e.g. `GET /api/v1/warehouses/55`), `TenantSecurityService.validateTenantOwnership()` throws a `TenantAccessDeniedException` (HTTP 403), guaranteeing strict tenant data isolation.

Q5 How is the Warehouse Hierarchy designed?

AeroWMS models physical warehouse spatial layout down 5 distinct hierarchical levels:

```
[1. Warehouse] → [2. Zone] → [3. Rack] → [4. Shelf] → [5. Bin (Location QR)]
```

- **Spatial Topology:** A physical Warehouse contains functional **Zones** (Cold Chain, Bulk Storage), which contain **Racks** (aisle bays), supporting **Shelves**, housing addressable **Bins**.
- **Hierarchical Location Code:** Each storage bin receives an auto-generated composite string:

`Location Code = [WarehouseCode]-[ZoneCode]-[RackCode]-[ShelfCode]-[BinCode] (e.g. WH01-ZA-R01-S02-B03)`
- **Dynamic QR Generation:** `WarehouseService.addBin()` formats this location code and calls Google ZXing (`QRCodeWriter`) to generate a Base64 PNG image stored directly on the bin record, enabling instant location verification via shelf sticker scans.

Q6 How are Inventory and Stock Movements handled?

Inventory balances and movements follow double-entry ledger accounting principles:

- **Discrete Balance Matrix (Inventory):** Tracked per coordinate: (`clientId`, `warehouseId`, `binId`, `productId`, `batchId`) with quantity (physical stock) and `reservedQuantity` (in-transit stock).
- **Double-Entry Transaction Ledger:** Direct balance edits are prohibited. Every modification writes an immutable `StockTransaction` logging `transactionType` (`STOCK_IN`, `STOCK_OUT`, `TRANSFER_IN`, `TRANSFER_OUT`, `ADJUSTMENT`), `balanceBefore`, `balanceAfter`, user ID, reference number, and timestamp.
- **ACID Atomicity:** Spring's `@Transactional` ensures that updating balances and writing the ledger commit together or roll back entirely.
- **Negative Stock & Low Stock Rules:** The service validates `quantity >= requestedQty` before deducting stock, and triggers automatic low-stock notifications when aggregate tenant stock drops below `reorderLevel`.

Q7 How does the FEFO (First-Expired-First-Out) logic work?

FEFO is crucial in grocery, pharmaceutical, and chemical supply chains to prevent inventory loss from expiration:

- **Inbound Batch Registration:** During Stock In, staff enter a batchNumber, mfgDate, and expiryDate, persisted in the ProductBatch collection.
- **Outbound Recommendation Algorithm:** When Stock Out is initiated, InventoryService.getFefoRecommendations() executes an ascending sort query on MongoDB:

```
productBatchRepository.findByIdAndProductIdOrderByExpiryDateAsc(clientId, productId);
```

- **Batch Prioritization:** Batches with the earliest expiration dates are recommended first. The UI displays days remaining and exact bin locations, ensuring aging inventory is shipped before newer stock.

Q8 How are Stock Transfers processed?

Inter-warehouse transfers follow a **4-stage finite state machine** to prevent stock race conditions:

[1. PENDING] → [2. APPROVED] → [3. DISPATCHED] → [4. RECEIVED] (Stock Reserved) (Manager Review) (Source Deducted) (Destination Added)

- **1. Creation (PENDING):** Validates source and destination warehouses belong to the client. Source stock is **reserved** (`reservedQuantity += qty`) so it cannot be double-sold locally while awaiting review.
- **2. Approval (APPROVED):** Facility manager authorizes the transfer request.
- **3. Dispatch (DISPATCHED):** Goods leave source. Physical inventory is deducted: `quantity -= qty` and `reservedQuantity -= qty`. A `TRANSFER_OUT` transaction is recorded.
- **4. Receipt (RECEIVED):** Destination warehouse staff inspect and assign items into destination bins. Destination inventory increments (`quantity += qty`) and a `TRANSFER_IN` transaction is committed.

Q9 How do QR and Barcode scanning work?

AeroWMS features a dual-mode scanning architecture implemented in `scanner.component.ts`:

- **Optical Camera Viewport:** Integrates `html5-qrcode` to access device cameras via `HTML5 navigator.mediaDevices.getUserMedia`. An animated laser viewfinder processes canvas video frames in real time to decode 1D barcodes and 2D QR codes without native mobile apps.
- **Industrial Laser Scanner Gun Support:** Handheld USB/Bluetooth scanners operate as keyboard wedge (HID) devices, rapidly typing characters and firing an Enter key event. The component listens to (`keyup.enter`), executes product lookup, and auto-refocuses the input for hands-free continuous scanning.
- **Web Audio API Telemetry:** Emulates industrial Honeywell/Zebra scan chimes by synthesizing an instant ****1760 Hz sine wave tone**** for 120ms with exponential gain decay using the native browser `AudioContext`.
- **Barcode Label Generator:** `QrBarcodeService` uses `ZXing (Code128Writer)` to generate Base64 PNG barcode labels, rendered into printable price stickers on demand.

Q10 How is the Database structured?

Runs on **MongoDB Atlas** using a relational document modeling approach with sequential numeric IDs:

- **Sequential Numeric IDs:** An atomic counter collection (`DatabaseSequence`) generates clean integer primary keys (1, 2, 3) alongside MongoDB's internal `_id`.
- **Core Collections:** `clients`, `users`, `warehouses`, `zones`, `racks`, `shelves`, `bins`, `products`, `product_batches`, `inventory`, `stock_transactions`, `stock_transfers`, and `audit_logs`.
- **Compound Indexes:**

```
// Unique stock coordinate constraint (prevents duplicate inventory slots):
db.inventory.createIndex({ clientId: 1, warehouseId: 1, binId: 1, productId: 1, batchId: 1 }, { unique: true });
// Unique tenant SKU:
db.products.createIndex({ clientId: 1, sku: 1 }, { unique: true });
// Fast FEFO batch sorting:
db.product_batches.createIndex({ clientId: 1, productId: 1, expiryDate: 1 });
```

Q11 What Security Measures did you implement?

- **Password Security:** Hashed with salted BCryptPasswordEncoder (strength factor 10). Plaintext passwords are never stored or logged.
- **Stateless Authentication:** Validated cryptographically via HMAC-SHA256 signed JWT tokens on every HTTP call.
- **Injection Protection:** Spring Data MongoDB repository criteria eliminate NoSQL injection risks; Angular's template binding contextually encodes all data to prevent XSS.
- **Tenant & Branch Access Controls:** TenantSecurityService programmatically verifies tenant ownership on all resource lookups.
- **Immutable Audit Logging:** Administrative actions, inventory edits, deletions, and user logins are written to an append-only AuditLog collection recording user ID, action, IP address, and timestamp.

Q12 What was the most challenging part of the project?

The Challenge: Managing **High-Concurrency Inventory Consistency & Anti-Negative Stock Rules in a Multi-Tenant Architecture**.

Why It Was Difficult: In busy warehouses, multiple pickers simultaneously dispatch items from the same bin. If two workers attempt to dispatch the remaining stock simultaneously, a classic race condition could result in negative stock, duplicate allocation records, or ledger discrepancies without locking other tenants.

The Solution:

1. Wrapped all inventory mutations in Spring's `@Transactional` boundaries to guarantee atomic commits between balance updates and ledger transactions.
2. Introduced the **Stock Reservation Pattern** (`reservedQuantity`) during inter-warehouse transfers so inventory is locked upon request and cannot be double-allocated.
3. Calculated *effective available quantity* (`quantity - reservedQuantity`) before fulfilling any outbound request.
4. Configured MongoDB compound unique index constraints on (`clientId`, `warehouseId`, `binId`, `productId`, `batchId`) to physically reject duplicate parallel allocations at the database engine level.